

SageSched: 应对需求不确定性与混合资源需求的高效 LLM 调度

Zhenghao Gan¹ Yichen Bao¹ Yifei Liu¹ Chen Chen¹ Quan Chen¹ Minyi Guo¹

摘要

高效 LLM 推理调度对用户体验十分重要。不过, 推理需求具有明显不确定性, 输出长度事先未知; 还具有混合资源性, 同时大量消耗计算与内存。现有调度器依赖简单启发式, 或只考虑计算资源, 性能因而不够理想。

我们提出 SageSched, 有效处理这两种需求特征。它结合提示词内容与历史推理结果, 以轻量、准确的方式预测输出长度分布; 同时综合计算与内存, 为请求建立实际服务成本模型; 最后依据成本分布, 采用感知不确定性的调度策略, 改善整体效率。不同配置的真实平台实验表明, 其效率提升超过 28.7%。

1. 引言

大语言模型 (LLM) 推动了从会话助手 (OpenAI, 2024; DeepSeek, 2024) 到智能体与具身系统 (Chen et al., 2023; Wang et al., 2024) 等领域的变化。随着广泛使用, GPU 服务端经常需要并发处理大量推理请求 (Sun et al., 2024)。为改善端到端用户体验, 调度时越来越需要降低整体服务延迟, 即最后 token 到达时间 (TTLT) (Shahout et al., 2025; Qiu et al., 2024)。

与传统工作负载相比, LLM 资源需求有两个挑战: 不确定性与混合资源性。一方面, 由于自回归生成, 推理输出长度事先无法确定, 具有强不确定性 (Yu et al., 2022); 操作系统 (Lozi et al., 2016) 或大数据 (Ousterhout et al., 2015) 负载则经常重复出现, 资源需求较稳定。另一方面, 推理涉及大量矩阵操作, 并高度依赖 KV 缓存 (Qin et al., 2024), 同时消耗大量计算与内存; 相比之下, 借助虚拟内存 (Abrossimov et al., 1989), 传统操作系统进程调度主要考虑计算成本。

面对这两种需求特征, 现有调度器难以达到高效率。vLLM (Kwon et al., 2023) 和 SGLang (Zheng et al., 2024) 等主流框架, 使用经典先来先服务, 经常因队头阻塞提高整体 TTLT。近期方法 (Shahout et al., 2025; Qiu et al., 2024; Fu et al., 2024) 根据预测输出长度近

似最短作业优先, 但仍不理想。首先, 它们通常微调模型, 直接预测输出长度; 训练与推断成本较高, 方法较重, 而模拟原生成结果又困难, 预测也可能不准确。

此外, 按输出 token 长度排序, 忽略混合资源需求, 无法表达实际服务成本。以单个预测均值作为排队指标, 也丢失了不确定性的分布信息, 难以实现最好的排队效果。

我们设计 SageSched, 通过三项技术处理这些问题。第一, 感知语义的历史预测器。提示词相似性与输出长度相似性相关, 因此结合提示词内容和历史推理结果预测长度分布, 无需为每个 LLM 维护重型预测模型, 也无需直接模拟完整生成过程。第二, 考虑计算与内存两方面的资源争用, 分别研究计算受限和内存受限的服务过程, 建立捕捉请求实际成本的统一模型。第三, 以分布描述服务成本, 采用感知不确定性的请求调度: 根据成本分布计算每个请求的 Gittins 指数。该指数对“时长未知、但时长分布已知”的作业, 能在相应理论设定下提供最优整体性能 (Gittins & Jones, 1979); 系统还周期性更新正在执行请求的指数, 使调度计划及时适应新信息。

我们在 vLLM 上实现系统 (vLLM Project, 2024), 使用多种真实 LLM 轨迹评估。相对先进调度器, 平均 TTLT 改善超过 28.7%。进一步分析分别验证预测、成本建模与调度技术的优势; 大规模模拟也表明, 它在较大集群中可扩展, 调度开销较低。

本文贡献如下:

- 通过真实平台测量, 指出现有调度器面对需求不确定性与混合资源性时的限制。
- 设计 SageSched, 结合感知语义的历史预测、根据资源瓶颈建立的成本模型, 以及感知不确定性的请求排队, 降低 TTLT。
- 通过真实平台实验与模拟, 验证系统显著提高效率, TTLT 改善超过 28.7%。

2. 背景与研究动机

2.1. LLM 调度问题

推理需求的不确定性与混合资源性。 LLM (DeepSeek, 2024; Meta AI, 2024b) 已广泛用于聊天 (OpenAI,

¹ 上海交通大学。通讯作者: Chen Chen <chen-chen@sjtu.edu.cn>。

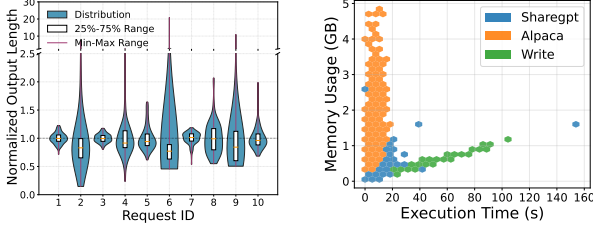


图 1. LLM 推理资源需求具有不确定性与混合资源性的实验证据。图内文字译释: 图内: Distribution = 分布, 25%-75% Range = 第 25 至第 75 百分位区间, Min-Max Range = 最小值至最大值区间, Request ID = 请求编号, Normalized Output Length = 归一化输出长度; Execution Time (s) = 执行时间 (秒), Memory Usage (GB) = 内存占用 (GB), Sharegpt、Alpaca、Write 为数据集名称。

图 1(a) 表明, 即使提示词固定, 不同运行也可能生成不同输出。¹因此, 最终输出长度只能在完成后知道, 具有强不确定性。

此外, 需求还具有混合资源性。推理通常在 GPU 保存 token 序列的 KV 缓存 (Qin et al., 2024), 避免重新计算; vLLM 等框架中, 分配这部分内存是推进推理的前提 (vLLM Project, 2024)。

图 1(b) 分析三个典型数据集的请求内存占用与执行时间, 它们依照文献 (Hu et al., 2024b) 选择: *SharedGPT* (Hugging Face Datasets, 2025b)、*Alpaca-Summarization* (Hugging Face Datasets, 2025a) 和 *Document-Write* (Hugging Face Datasets, 2025c)。结果表明, 计算和内存存在服务中都应是首要资源, 而传统操作系统工作负载主要按计算成本调度 (GeeksforGeeks, 2026)。²

服务后端的调度十分重要。 不同用户的请求经常共用后端, 争用有限计算与内存 (Recasens et al., 2025)。

为改善用户体验, 需要妥善调度, 降低整体延迟。推理具有首 token 时间 (TTFT)、每输出 token 时间 (TPOT)、最后 token 时间 (TTLT) 等指标; 本文主要关注综合性更强的 TTLT。³这一选择与已有研究一

¹本文所有推理温度均设为 0.6, 这是许多常用 LLM 的默认设置 (AI, 2024; Qwen, 2024)。

²译注: 原稿的数据集名称多处写作 SharedGPT, 但所引用数据源为 ShareGPT; 保留原稿命名, 并说明这一拼写差异。“操作系统主要按计算调度”是本文用于比较的概括, 不意味着实际系统完全不考虑内存约束。

³首先, TTLT 对智能体 AI 等场景日益重要, 例如 LLM 机器人控制 (Chen et al., 2023; Wang et al., 2024), 必须等

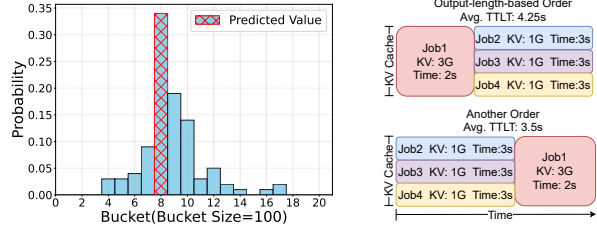


图 2. 现有调度器面对需求不确定性和混合资源特征时的不足示例。图内文字译释: 图内: Bucket (Bucket Size=100) = 桶编号 (桶宽 100 token), Probability = 概率, Predicted Value = 预测值; KV Cache = KV 缓存, Output-length-based Order = 按输出长度排序, Another Order = 另一种排序, Avg. TTLT = 平均 TTLT, Time = 时间, Job = 作业。作业 1 占用 3G、执行 2 秒, 作业 2-4 各占用 1G、执行 3 秒; 两种排序的平均 TTLT 分别为 4.25 秒与 3.5 秒。

致 (Shahout et al., 2025; Qiu et al., 2024)。⁴

2.2. 现有调度器的限制

已有研究提出了多种 LLM 调度器。面对不确定性, 早期方法通常采用不感知需求的启发式。vLLM (Kwon et al., 2023)、SGLang (Zheng et al., 2024) 使用简单 FCFS, 存在队头阻塞。FastServe (Wu et al., 2023) 使用多级反馈队列, 在固定服务时间片后, 将持续执行的请求降到更低优先级, 实质近似最短剩余处理时间 (SRPT), 但效果仍不及精确 SRPT, 并引入抢占开销。

近期方法转向基于预测的调度。SSJF (Qiu et al., 2024) 用微调 BERT-base, 根据提示词预测长度, 近似最短作业优先 (SJF)。Fu 等 (Fu et al., 2024) 使用 OPT-125M, 预测请求的相对长度排序, 而非精确长度。TRAIL (Shahout et al., 2025) 使用 MLP, 读取部分 LLM 层的特征嵌入, 在每次迭代连续预测长度, 以近似原文所称 “SPRT”。⁵

这些方法处理不确定性仍有不足。首先, 预测器需要较重的离线训练, 微调通常需要收集并处理超过一万条样本 (Jin et al., 2023; Fu et al., 2024); 针对一个 LLM 训练的模型, 也不能直接用于另一个模型。其次, 输出单个预测值会丢失有价值的概率分布。长度不确定性是推理的固有性质, 单值很难完全准确。图 2(a)

生成完成后才能启动下游操作。其次, 优化 TTLT 经常有助于其他指标: 缓解队头阻塞可自然改善 TTFT, 后文实验亦予以验证; 对于 TPOT, 相关统计分析常采用 “TTLT 除以输出 token 数” 的定义 (Wu et al., 2023; Fu et al., 2024), 因而降低 TTLT 会相应降低这一归一化值。

⁴译注: 这里的 TTLT / 输出长度是按整次响应归一化的时间, 包含排队与预填充; 通常的流式 TPOT 则衡量首 token 之后的生成间隔, 不能无条件与此定义等同。

⁵译注: 此处 SPRT 应为 SRPT, 即最短剩余处理时间的字母顺序笔误; 保留并指出原文差异。

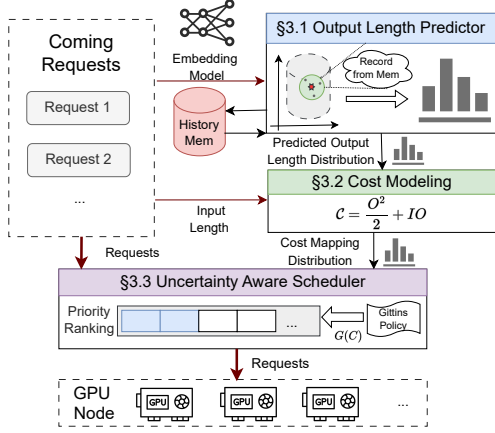


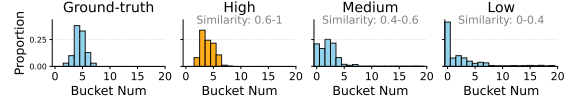
图 3. SageSched 的工作流程概览。图内文字译释：图内流程标签：Coming Requests = 新到达请求；Request 1/2 = 请求 1/2；Embedding Model = 嵌入模型；History Mem = 历史记录存储；Record from Mem = 从存储检索记录；Output Length Distribution = 预测输出长度分布；Input Length = 输入长度；Cost Modeling = 成本建模；Cost Mapping Distribution = 成本映射后的分布；Uncertainty Aware Scheduler = 感知不确定性的调度器；Priority Ranking = 优先级排序；Gittins Policy = Gittins 策略；Requests = 请求；GPU Node = GPU 节点。

中，文献 (Qiu et al., 2024) 所用 DistillBert 的单值预测，在总共 20 个长度桶中，只有 34.1% 的样本命中正确桶。第 3.3 节将说明，保留长度分布能改善调度。因此，需要无需训练、直接预测分布的方法。

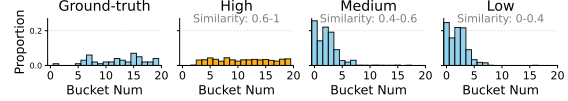
这些调度器处理混合资源性也有不足：以输出长度表示计算时间，作为唯一服务成本，忽略 GPU 内存消耗，因而不准确。输入与输出 token 都占用 KV 空间，所以相同输出长度的请求，内存成本也可能差异很大。图 2(b) 表明，后端受内存限制时，只优先执行短输出请求可能不理想。因此，需要同时考虑计算与内存消耗。由于这两种需求特征，现有调度器难以达到较高整体效率。下面我们分别优化长度预测、成本建模和请求排队，设计新的调度器。

3. SageSched 的设计

本文设计了新的 LLM 调度器 SageSched，通过妥善处理需求的动态性、不确定性和混合资源特征，提高未 token 延迟 (TTLT) 的性能。图 3 展示了其整体工作流程。对于每个新到达的 LLM 推理请求，SageSched 首先使用轻量且准确的预测器预测输出长度分布 (第 3.1 节)；随后根据预测的输出长度，同时考虑内存与计算资源，量化请求的整体服务成本 (第 3.2 节)；最后采用感知不确定性的调度算法确定运行时请求排队顺序，以获得最优效率 (第 3.3 节)。



(a) 从 Alpaca 数据集采样的提示 1



(b) 从 Write 数据集采样的提示 2

图 4. 提示相似度更高的历史请求，能够更好地近似当前请求的输出长度分布。图内文字译释：图内：Ground-truth = 真实分布，High/Medium/Low = 高/中/低相似度组，Similarity = 相似度；对应区间分别为 0.6-1、0.4-0.6 与 0-0.4；Bucket Num = 桶编号，Proportion = 比例。

3.1. 基于历史记录语义感知预测器

如第 2.2 节所述，LLM 解码过程具有内在不确定性，因此，要准确预测输出 token 长度，预测方法必须给出分布，而不是单个数值。一种直接的做法是修改现有预测模型 (如 DistillBert (Hugging Face, 2024)) 的输出投影层，使其预测分布；这种定制模型实质上试图根据输入提示的语义，直接近似最终的生成结果。

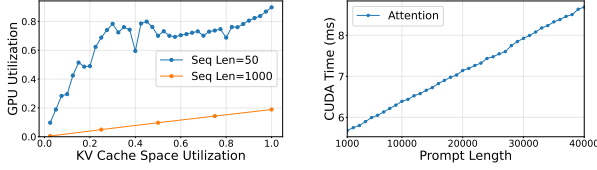
然而，维护这类预测模型仍然成本高昂，因为需要训练；而且不够准确，因为近似真实推理结果本身就很复杂。

事实上，我们发现，给定推理提示后，参考近期历史中的相似请求更适合预测输出长度，而不是用微调模型直接模拟内部生成过程。自回归生成十分复杂，相比直接模拟生成结果，判断提示的相似度容易得多。此外，已有工作 (Gong et al., 2025) 指出，相邻滑动时间窗口中的请求输出长度总体分布相对稳定，说明近期服务历史具有很高的参考价值。因此，选取提示相似的历史请求，有望让需求预测同时变得更容易、更准确。

为检验这一点，我们通过实验研究提示相似度与输出长度相似度之间的关联。我们分别从 Alpaca 和 Write 数据集 (见第 2.1 节) 各采样一个提示。对每个提示，先执行 100 次推理，得到作为预测目标真实值的输出长度分布。随后，根据历史请求与所选提示的相似度，将历史请求分为三类；这里使用两个请求提示嵌入之间的余弦相似度衡量相似程度。如图 4 所示，在两种情况下，提示相似度与输出长度分布的相似程度都存在明显关联：相似度更高的分组，其分布曲线形状更接近目标分布。

基于上述分析，我们提出基于历史记录语义感知预测器，将提示内容与历史执行情况结合，直接预测输出长度分布。具体而言，我们记录近期已服务请求的推理信息，包括提示内容和输出长度。对于每个新请求，我们检索⁶提示足够相似的历史请求 (相似度阈值

⁶历史窗口包含 10,000 条记录，并按先进先出 (FIFO) 方



(a) 批大小增加时 GPU 利用率与 KVCache 占用量的关系。 (b) 解码推进时，每一步注意力计算时间的变化。

图 5. 瞬时资源瓶颈以及解码计算成本特征的测量。所有测量均在 H800 GPU 上使用 Qwen3-32B 模型进行。图内文字译释：图内：GPU Utilization = GPU 利用率，KV Cache Space Utilization = KV 缓存空间利用率，Seq Len = 序列长度；Attention = 注意力模块，Prompt Length = 提示长度，CUDA Time (ms) = CUDA 时间（毫秒）。

默认为 0.8)，并以这些请求的输出长度分布作为预测结果。该方法不需要微调任何模型，因此适用于不同 LLM；也不需要模拟复杂的生成过程。由此，我们能够以轻量方式准确预测输出长度分布。

3.2. 基于资源瓶颈的成本建模

采用上述预测方法后，我们可以得到每个即将到来的请求的输出长度分布。然而，如第 2.2 节所述，需求具有混合资源特征，直接将输出 token 长度作为 LLM 请求的服务成本并不充分。本节需要建立同时考虑计算与内存资源的综合成本模型。

我们注意到，构建多资源成本模型时，首先必须判断哪类资源构成真正的瓶颈，因为这决定后续成本分析的方向。实际 LLM 服务后端可能受计算能力限制，也可能受内存容量限制，具体取决于运行时负载特征。图 5(a) 展示了⁷ H100 GPU 上一次解码步骤的计算和内存消耗随批大小增加的关系。我们分别将序列长度（输入长度加上当前已生成的输出长度）固定为 50 和 1000。较长序列通常更占用内存：处理短序列时，GPU 内存尚未达到上限，计算利用率就已很高；而处理长序列时，GPU 内存已满，计算利用率却还没有上升。只有某类资源成为瓶颈时，该资源上的服务成本才起决定作用。因此，我们分别分析这两种状态下的 LLM 服务成本。

首先考虑内存受限状态下的推理成本。在这种情况下，LLM 请求的 KVCache 占用量是成本建模的关键：较低的 KVCache 消耗意味着能够并发服务更多请求；当 GPU 计算利用率较低时，增加并发请求不会提高一次迭代的执行时间。因此，内存受限时，应主要考察一个

式持续更新。当高度相似请求不足时，例如在预热阶段，我们将公共数据集中的请求加入检索集合。我们使用高效的 FAISS IndexFlat (Meta AI, 2024a) 工具进行嵌入检索，通常耗时小于 1 ms。

⁷在图 5 中，我们通过实际达到的 TFLOPs 与峰值 TFLOPs 的比值估计 GPU 利用率，并使用 torch.profiler 测量 CUDA 时间。

请求在整个推理过程中累计占用的 KVCache。

设一次推理的输入（即提示）和输出 token 长度分别为 I 和 O ，则其总服务成本 C 可表示为 $C_M = \sum_{l=I}^{I+O} l * U_{MT} = (\frac{O^2}{2} + IO)U_{MT}$ ，其中 U_{MT} 是单位内存与时间的乘积，即单个 token 在单个步骤中占据的 KVCache 空间。⁸

接下来考虑计算受限状态。此时，LLM 推理的 KVCache 占用量不是服务后端的主要约束，只需量化整个推理生命周期中的总计算量。

为此，在图 5(b) 中，我们测量了单独服务一次推理时，不同解码步骤⁹中的单次迭代注意力计算时间；注意力是 Transformer 中的关键计算模块。结果证实，该时间与当前累计序列长度呈线性关系。

因此，单次推理的总计算成本可以表示为 $C_C = \sum_{l=I}^{I+O} l * U_{CT} = (\frac{O^2}{2} + IO)U_{CT}$ ，其中 U_{CT} 是单位计算与时间的乘积，即单个步骤处理一个 KVCache token 所需的平均计算量。

上述分析表明，计算成本与内存成本遵循相同形式；两者单位不同，但不影响请求之间的相对成本顺序。因此，在实践中可以统一建模，无须随瓶颈类型切换： $C = \frac{O^2}{2} + IO$ 。虽然这种形式在两类资源之间一致，但它与已有公式有实质区别，例如文献 (Qiu et al., 2024; Fu et al., 2024; Shahout et al., 2025) 仅使用 O ，而文献 (Sheng et al., 2024) 使用 I 与 O 的简单加权和。第 4.3.2 节的实验将验证该建模方法的优势。

3.3. 感知不确定性的请求调度

结合上述输出预测和成本建模方法，我们能够在每个 LLM 请求到达时描述其成本分布。这可以缓解需求不确定性，但无法完全消除不确定性。因此，需要设计感知不确定性的调度算法，以获得理论上最优的 TTLT 性能。

安排请求队列时，需要从每个请求的成本分布中计算合适的调度指数；图 6 表明，均值并不是最优选择。我们发现，选择优先级最高的推理进行服务，类似于在多臂老虎机问题 (Mahajan & Teneketzis, 2008) 中选择最有回报的臂：成本分布实质上描述了向请求分配额外资源后，获得推理完成这一回报的可能性。对于回报不确定、但回报分布已知的多臂老虎机问题，已有研究表明，Gittins 策略 (Gittins & Jones, 1979) 可以达到理论上的最优性能。

⁸译注：原文将离散求和与二次表达式直接写成等号；后者可理解为忽略端点及线性项的连续近似。此处保留原文公式。

⁹本文主要考察解码步骤的资源消耗。随着推理型 LLM (Cai et al., 2025) 广泛应用，推理的解码阶段往往主导 TTLT。此外，除随序列长度线性增加的注意力计算时间外，还存在与序列长度无关的固定 FFN 计算时间，以及 CUDA 启动延迟等较小因素。不过，得益于 GPU GEMM 优化，较长解码和较大批大小能够显著摊薄 FFN 时间，因此为简化分析，此处将其忽略。

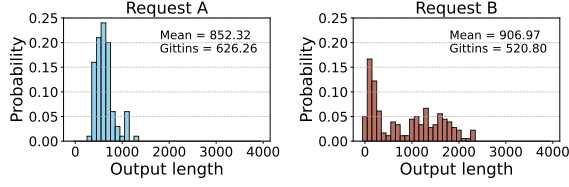


图 6. 仅按均值确定请求优先级的局限性示例。虽然请求 B 的均值更大, 但由于其需求分布形状不规则, 先服务它反而有望获得更好的平均 TTLT。图内文字译释: 图内: Request A/B = 请求 A/B; Output length = 输出长度; Probability = 概率; Mean = 均值; Gittins = 本文定义的 Gittins 指数。A 的均值/指数为 852.32 / 626.26, B 为 906.97 / 520.80; 本文的指数越小优先级越高。

具体而言, 对于成本分布为 $\mathcal{D}$ 的请求, Gittins 策略计算其 Gittins 指数, 定义为 $G(\mathcal{D}) = \inf_{\Delta > 0, X \sim \mathcal{D}} \frac{E[\min\{X, \Delta\}]}{P(X \leq \Delta)}$; 调度时优先服务指数较小的请求。在该公式中, $E[\min\{X, \Delta\}]$ 表示分配服务预算 Δ 时, 请求消耗的服务量; $P(X \leq \Delta)$ 表示预算 Δ 耗尽前请求成功完成的概率。因此, G 表示该请求可以达到的最小摊销成本。已有研究证明, 依据这种 Gittins 指数而非分布期望来调度作业, 可以获得最小平均延迟 (TTLT) (Gittins, 1989)。图 6 展示了其效果: Gittins 策略优先服务近期更有可能完成的请求。¹⁰

此外, 推理开始后, 还应在运行时适当更新每个待处理请求的 Gittins 指数。如 TRAIL (Shahout et al., 2025) 和 FastServe (Wu et al., 2023) 所述, 为优化平均 TTLT, 需要允许抢占; 采用交换与计算重叠、KVCache 预取等近期技术, 可以将抢占开销控制得很低。理想情况下, 应根据剩余服务成本分布持续刷新每个活跃推理的 Gittins 指数。然而, 每个推理步骤之后都刷新并不现实: 监控开销较高, 还可能发生抖动, 即 Gittins 指数相近的两个请求, 其相对调度次序频繁反转。因此, 我们将每个请求的成本范围划分为多个桶, 默认 10 个; 只在跨越桶边界时刷新指数, 以平衡重新调度的及时性与系统稳定性。

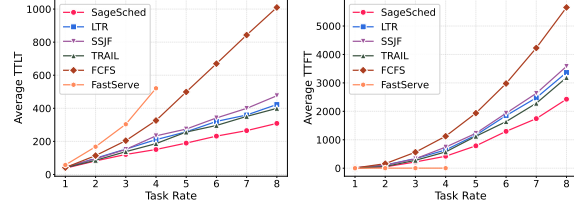
4. 实验评估

4.1. 实验设置

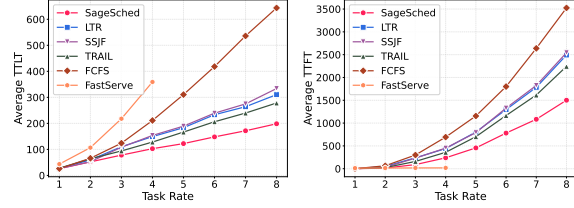
硬件平台。 真实平台实验分别使用 Llama3.1-8B (AI, 2024) 和 Qwen3-32B (Qwen, 2024) 作为 LLM 后端。Llama3.1-8B 部署在配备 A40-PCIe-48GB GPU 的服务器上, Qwen3-32B 部署在配备 H800-PCIe-96GB GPU 的服务器上。此外, 我们构建了最多包含 64 个 GPU 的模拟器, 以评估 SageSched 的可扩展性。

工作负载。 实验采用文献 (Hu et al., 2024b) 中的三个典型推理数据集: SharedGPT (Hugging Face

¹⁰译注: 这里沿用原文的理论最优性表述; 经典结论依赖其调度模型的假设, 本文没有给出对批处理 GPU、预测误差及换入换出成本同时成立的最优性证明。



(a) Llama3.1-8B



(b) Qwen3-32B

图 7. 混合数据集上的端到端性能。图内文字译释: 四幅图中: Task Rate = 任务到达率; Average TTLT = 平均未 token 延迟; Average TTFT = 平均首 token 延迟; 各调度器名称保持原样。原图纵轴没有标出时间单位。

Datasets, 2025b)、Alpaca-Summarization (Hugging Face Datasets, 2025a) 和 Document-Write (Hugging Face Datasets, 2025c)。图 1(b) 已说明这些数据集的特征。请求到达采用不同 λ 参数下的泊松分布, 从而生成不同任务到达率。

基线。 我们将 SageSched 与多个代表性基线比较: FCFS (Kwon et al., 2023)、FastServe (Wu et al., 2023)、SSJF (Qiu et al., 2024)、LTR (Fu et al., 2024) 和 TRAIL (Shahout et al., 2025)。这些基线均已在第 2.2 节介绍, 其超参数均采用默认值。对于 SageSched, 默认将预测器的选择比例设为 0.8, 将 Gittins 分析的桶大小设为 200。¹¹

评价指标。 如第 2.1 节所述, 我们以平均 TTLT 作为主要调度指标, 同时评估各调度器的平均 TTFT。

4.2. 端到端实验

混合数据集请求实验。 端到端评估首先合并三个数据集, 再随机提交其中的请求, 分别测试不同任务到达率。如图 7 所示, SageSched 在全部测试配置中都获得最佳 TTLT 调度效率。例如, 以每秒 8 个请求 (RPS) 向 Qwen3-32B 提交推理时, SageSched 比次优调度器 TRAIL 提升 28.7%。当资源竞争更激烈时, 提升更大, 因为 SageSched 能够妥善处理 LLM 推理需求的不确定性和混合资源特征, 持续确定较好的调度顺序。

¹¹译注: 此处原文写作 TRIAL, 结合引文和下文, 应指 TRAIL。设计部分将 0.8 称为相似度阈值, 并称默认划分 10 个桶; 实验部分采用“桶大小 200”的表述。原稿未明确两种桶参数的换算关系, 此处分别保留。

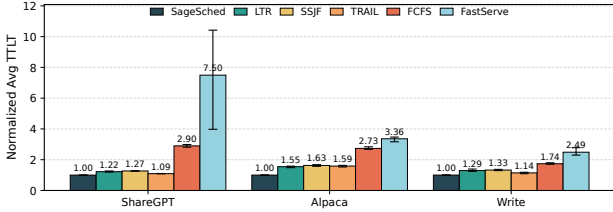


图 8. 分别采用各个数据集时的端到端性能。图内文字译释：图内：Normalized Avg TTLT = 归一化平均 TTLT；ShareGPT、Alpaca、Write 是数据集名称；各组均以 SageSched 的 TTLT 归一化为 1.00。

虽然 TTFT 不是本文的主要目标，但图 7 表明，*SageSched* 也能获得令人满意的 TTFT，因为它能有效缓解队头阻塞。特别地，FastServe 始终优先处理新到达的请求，因此 TTFT 最好；但其多级反馈队列 (MLFQ) 交错执行方式，使 TTLT 明显差于其他调度器。在这些比较中，*SageSched* 是唯一同时在 TTLT 和 TTFT 上表现良好的调度器。

各数据集单独请求实验。 三个数据集的输入与输出长度分布不同 (图 1(b))，因此我们分别测试，结果见图 8。*SageSched* 在每个数据集上都持续表现最好。尤其是在 Alpaca (摘要) 数据集上，其优势最明显，因为该数据集通常具有较长输入，而现有仅按输出长度调度的方法对这类请求的成本建模非常不准确。

4.3. 各组件的深入分析

本节深入检验 *SageSched* 各组件的有效性。除非另有说明，均沿用图 7 的设置，请求到达率为 8 RPS。

4.3.1. 预测器设计的优势

针对传统输出长度预测器的不足，第 3.1 节提出了新的基于历史记录语义感知预测器。这里从端到端 TTLT 性能检验其必要性。具体而言，增加两种预测基线：(1) 不感知语义的历史预测器，参考输入长度相近的历史请求预测当前请求的输出长度，采用与我们相同的筛选方法；(2) 基于 LLM 的语义感知预测器，沿用文献 (Qiu et al., 2024) 的 DistillBert 模型，但删除 logits.argmax 层，以输出分布。图 9 展示 *SageSched* 使用不同预测器的性能。由于分布预测更准确，我们的语义感知历史预测器使 *SageSched* 获得最佳 TTLT。

此外，我们的预测器在训练和预测开销方面也优于其他方案。首先，该方法完全消除了训练开销；基于 LLM 的预测器则需要数小时构建训练数据集，再用数十分钟微调模型。其次，我们的平均单请求预测延迟低于 0.5 ms，其中语义嵌入提取为 0.22 ms，向量检索为 0.15 ms；而基于 LLM 的预测器约需 3.6 ms。这些结果共同证实，语义感知历史预测器既轻量又准确。

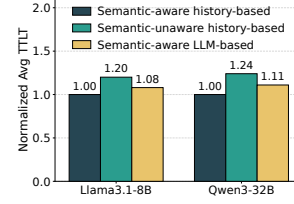


图 9. 不同输出长度预测方法的性能比较。图内文字译释：图内：Normalized Avg TTLT = 归一化平均 TTLT；Semantic-aware history-based = 语义感知历史预测；Semantic-unaware history-based = 不感知语义的历史预测；Semantic-aware LLM-based = 基于 LLM 的语义感知预测。

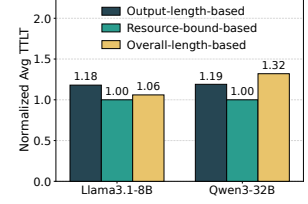


图 10. 不同服务成本建模方法的性能比较。图内文字译释：图内：Normalized Avg TTLT = 归一化平均 TTLT；Output-length-based = 基于输出长度；Resource-bound-based = 基于资源瓶颈；Overall-length-based = 基于总长度。

4.3.2. 成本建模方法的优势

在第 3.2 节中，我们分别分析计算受限与内存受限情况，建立了描述 LLM 推理成本的基于资源瓶颈的模型。为验证其必要性，我们将其分别替换为两种方法：(1) 基于输出长度的模型，直接以输出长度作为推理成本 (Shahout et al., 2025; Qiu et al., 2024)；(2) 基于总长度的模型，结合输入和输出长度计算成本，其中按照文献 (Sheng et al., 2024)，给予输出长度两倍权重。图 10 展示不同模型下的整体 TTLT，证实采用基于资源瓶颈的成本模型是必要的。

4.3.3. 调度策略的优势

第 3.3 节的感知不确定性调度算法有两项关键技术：基于 Gittins 指数排序以及运行时刷新 Gittins 指数。这里分别评估两者的收益，并检验预测不准确时的调度稳健性。我们先增加两个基线：(1) Mean，根据请求成本分布的均值排序；(2) Gittins-，采用 Gittins 策略，但不在桶边界及时刷新指数。此外，对每种调度器，我们还测试预测不准确的成本分布：按照 1:4 的权重比例，将均匀分布混入原分布。如图 11 所示，Gittins 指数排序与运行时刷新均有助于改善 TTLT；加入噪声时，基于 Gittins 指数的算法性能下降幅度也远小于其他基线。

4.4. 开销与敏感性分析

本节研究 *SageSched* 的可扩展性，并分析所引入超参数的敏感性。除非另有说明，采用图 7 的实验设置。

可扩展性。 为评估 *SageSched* 的可扩展性，我们模拟最多 64 个 GPU 节点的大规模集群部署，系统负载 (RPS) 随集群规模同比增加。平均每个节点每秒服务 8 个请求，队列中最多缓冲 1,000 个请求。我们假设输出长度固定为 1,000 token，测量预测与调度阶段带来的单请求延迟。图 12 表明，该延迟仅随集群规模线性

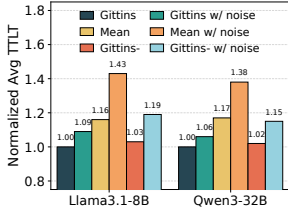


图 11. 加入成本噪声时, 不同调度方法的比较。图内文字译释: 图内: Normalized Avg TTLT = 归一化平均 TTLT; Mean = 按均值排序; Gittins = 不及时刷新指数的 Gittins 策略; w/ noise = 加入噪声; Gittins 表示完整策略。

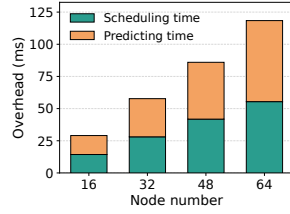
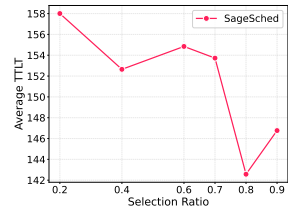
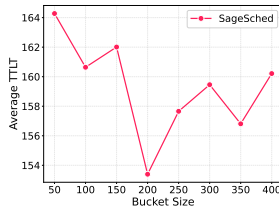


图 12. 不同规模下 SageSched 的预测与调度开销。图内文字译释: 图内: Node number = 节点数; Overhead (ms) = 开销 (毫秒); Scheduling time = 调度时间; Predicting time = 预测时间。



(a) 预测器超参数



(b) 调度器超参数

图 13. SageSched 超参数的敏感性分析。图内文字译释: 图内: Selection Ratio = 选择比例; Bucket Size = 桶大小; Average TTLT = 平均 TTLT; 曲线为 SageSched。

增长。即使在最大的 64 节点规模下, 每个请求平均额外延迟也保持在约 100 ms。相对于通常以数秒至数分钟为时间粒度的 LLM 推理工作负载 (如图 7), 这种开销很小。

此外, 对于大规模集群, 还可以部署多个并发调度器, 进一步降低上述开销。

相似度阈值。 预测器设计 (第 3.1 节) 采用余弦相似度阈值, 筛选提示相近的历史请求。这里分别设置不同阈值, 真实平台性能见图 13(a)。结果表明, 0.8 的性能最好: 阈值过高会使选中的请求过少, 削弱对需求不确定性的刻画; 阈值过低则会无关请求混入采样分布。

桶大小。 第 3.3 节还使用桶大小超参数, 控制 Gittins 指数的刷新频率。图 13(b) 测量 SageSched 在不同桶大小下的平均 TTLT。结果表明, 桶大小为 200 时性能最好, 因此采用该默认值。桶过小会增加优先级计算和重新调度开销, 桶过大则降低优先级分配的准确性, 二者都会损害整体效率。

5. 其他相关工作

许多研究从互补角度改善 LLM 服务效率, 可以与 SageSched 结合。例如, 一些工作在逐 token 层面加速

单次推理 (Dao et al., 2022; Dao, 2023; Dettmers et al., 2022; Frantar et al., 2022; Leviathan et al., 2023; Miao et al., 2023), 或优化部署方式 (Zhong et al., 2024; Qin et al., 2024; Hu et al., 2024a;b; Agrawal et al., 2024)。另一些工作通过重新设计批处理和准入控制机制, 提高后端利用率 (Duan et al., 2024; Xiang et al., 2025)。此外, KVCache 存储经常是主要瓶颈, 因此已有许多技术通过压缩或选择性保留, 减小 KVCache 大小 (Xiao et al., 2023; Zhang et al., 2023; Lee et al., 2024)。受篇幅限制, 详细介绍放在附录中。

6. 结论

本文提出 SageSched, 一个专门应对 LLM 工作负载不确定性及混合资源特征的高效调度器。它采用基于历史记录的语义感知预测器, 以轻量且准确的方式预测输出长度分布; 同时考虑计算和内存瓶颈, 建立 LLM 推理的整体服务成本模型; 并采用感知不确定性的调度策略, 使用 Gittins 指数改善排队性能。真实平台和模拟实验表明, SageSched 能够显著改善端到端效率, 使 TTLT 降低超过 28.7%。

参考文献

- Abrossimov, E., Rozier, M., and Shapiro, M. Generic virtual memory management for operating system kernels. In *Proceedings of the twelfth ACM symposium on Operating systems principles*, pp. 123–136, 1989.
- Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B., Tumanov, A., and Ramjee, R. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 117–134, 2024.
- AI, M. Llama-3.1-8b-instruct. <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>, 2024. Accessed: 2026-01-29.
- Cai, Z., Xiao, W., Sun, H., Luo, C., Zhang, Y., Wan, K., Li, Y., Zhou, Y., Chang, L.-W., Gu, J., et al. R-kv: Redundancy-aware kv cache compression for reasoning models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Chen, G., Yu, X., Ling, N., and Zhong, L. Typefly: Flying drones with large language model. *arXiv preprint arXiv:2312.14950*, 2023.
- Cursor. Cursor. <https://cursor.com/>, 2024. Accessed: 2026-01-27.

- Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- DeepSeek. Deepseek. <https://www.deepseek.com/>, 2024. Accessed: 2026-01-27.
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems*, 35: 30318–30332, 2022.
- Duan, J., Lu, R., Duanmu, H., Li, X., Zhang, X., Lin, D., Stoica, I., and Zhang, H. Muxserve: Flexible spatial-temporal multiplexing for multiple LLM serving. In *International Conference on Machine Learning (ICML)*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=R0SoZvqXyQ>.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- Fu, Y., Zhu, S., Su, R., Qiao, A., Stoica, I., and Zhang, H. Efficient llm scheduling by learning to rank. *Advances in Neural Information Processing Systems*, 37:59006–59029, 2024.
- GeeksforGeeks. Comparison of different cpu scheduling algorithms in operating systems. <https://www.geeksforgeeks.org/comparison-of-different-cpu-scheduling-algorithms-in-os/>, 2026. Accessed: 2026-01-27.
- Gittins, J. C. Multi-armed bandit allocation indices. *Wiley-Interscience Series in Systems and Optimization*, 1989.
- Gittins, J. C. and Jones, D. M. Bandit processes and dynamic allocation indices. *Journal of the Royal Statistical Society: Series B (Methodological)*, 41 (2):148–164, 1979.
- Gong, R., Bai, S., Wu, S., Fan, Y., Wang, Z., Li, X., Yang, H., and Liu, X. Past-future scheduler for llm serving under sla guarantees. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 798–813, 2025.
- Hu, C., Huang, H., Hu, J., Xu, J., Chen, X., Xie, T., Wang, C., Wang, S., Bao, Y., Sun, N., et al. Memserve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*, 2024a.
- Hu, C., Huang, H., Xu, L., Chen, X., Xu, J., Chen, S., Feng, H., Wang, C., Wang, S., Bao, Y., et al. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024b.
- Hugging Face. Distilbert. https://huggingface.co/docs/transformers/model_doc/distilbert, 2024. Hugging Face Transformers Documentation; Accessed: 2026-01-27.
- Hugging Face Datasets. Alpaca-pubmed summarization dataset. <https://huggingface.co/datasets/ZhongshengWang/Alpaca-pubmed-summarization>, 2025a. Accessed: 2026-01-27.
- Hugging Face Datasets. Sharegpt_vicuna_unfiltered dataset. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered, 2025b. Accessed: 2026-01-27.
- Hugging Face Datasets. write_doc_sft_v1 dataset. https://huggingface.co/datasets/lancexiao/write_doc_sft_v1, 2025c. Accessed: 2026-01-27.
- Jin, Y., Wu, C.-F., Brooks, D., and Wei, G.-Y. s^3 : Increasing gpu utilization during generative inference for higher throughput. *Advances in Neural Information Processing Systems*, 36: 18015–18027, 2023.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.
- Lee, W., Lee, J., Seo, J., and Sim, J. Infinigen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 155–172. USENIX Association, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/lee>.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine*

- Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Liu, X., Hu, L., Bailis, P., Stoica, I., Deng, Z., Cheung, A., and Zhang, H. Online speculative decoding. *CoRR*, abs/2310.07177, 2023. doi: 10.48550/ARXIV.2310.07177. URL <https://doi.org/10.48550/arXiv.2310.07177>.
- Liu, Z., Yuan, J., Jin, H., Zhong, S. H., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=L057s2Rq80>.
- Lozi, J.-P., Lepers, B., Funston, J., Gaud, F., Quéma, V., and Fedorova, A. The linux scheduler: a decade of wasted cores. In *Proceedings of the Eleventh European Conference on Computer Systems*, pp. 1–16, 2016.
- Mahajan, A. and Teneketzis, D. Multi-armed bandit problems. In *Foundations and applications of sensor management*, pp. 121–151. Springer, 2008.
- Meta AI. Faiss: A library for efficient similarity search and clustering of dense vectors. <https://ai.meta.com/tools/faiss/>, 2024a. Accessed: 2026-01-28.
- Meta AI. Llama. <https://www.llama.com/>, 2024b. Accessed: 2026-01-27.
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Wong, R. Y. Y., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. Specinfer: Accelerating generative LLM serving with speculative inference and token tree verification. *CoRR*, abs/2305.09781, 2023. doi: 10.48550/ARXIV.2305.09781. URL <https://doi.org/10.48550/arXiv.2305.09781>.
- OpenAI. Chatgpt. <https://openai.com/zh-Hans-CN/index/chatgpt/>, 2024. Accessed: 2026-01-27.
- Ousterhout, K., Rasti, R., Ratnasamy, S., Shenker, S., and Chun, B.-G. Making sense of performance in data analytics frameworks. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pp. 293–307, 2015.
- Qin, R., Li, Z., He, W., Cui, J., Tang, H., Ren, F., Ma, T., Cai, S., Zhang, Y., Zhang, M., et al. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage*, 2024.
- Qiu, H., Mao, W., Patke, A., Cui, S., Jha, S., Wang, C., Franke, H., Kalbarczyk, Z. T., Başar, T., and Iyer, R. K. Efficient interactive llm serving with proxy model-based sequence length prediction. *arXiv preprint arXiv:2404.08509*, 2024.
- Qwen. Qwen3-32b. <https://huggingface.co/Qwen/Qwen3-32B>, 2024. Accessed: 2026-01-29.
- Recasens, P. G., Agullo, F., Zhu, Y., Wang, C., Lee, E. K., Tardieu, O., Torres, J., and Berral, J. L. Mind the memory gap: Unveiling gpu bottlenecks in large-batch llm inference. In *2025 IEEE 18th International Conference on Cloud Computing (CLOUD)*, pp. 277–287, 2025. doi: 10.1109/CLOUD67622.2025.00036.
- Shahout, R., Malach, E., Liu, C., Jiang, W., Yu, M., and Mitzenmacher, M. Don’t stop me now: Embedding based scheduling for llms. In *ICLR*, 2025.
- Sheng, Y., Cao, S., Li, D., Zhu, B., Li, Z., Zhuo, D., Gonzalez, J. E., and Stoica, I. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 965–988, 2024.
- Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., and Lin, W. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 173–191. USENIX Association, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/sun-biao>.
- vLLM Project. vllm(v0.8.2). <https://github.com/vllm-project/vllm/tree/v0.8.2>, 2024. Accessed: 2026-01-27.
- Wang, G., Zhang, Y., Chen, H., Xiong, H., Wang, W., Li, L., Liu, T., Wang, Y., Wei, Y., and Guo, D. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Wang, S., Han, M., Jiao, Z., Zhang, Z., Wu, Y. N., Zhu, S.-C., and Liu, H. Llm³: Large language model-based task and motion planning with motion failure reasoning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 12086–12092. IEEE, 2024.
- Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., and Jin, X. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.

- Xiang, Y., Li, X., Qian, K., Yang, Y., Zhu, D., Yu, W., Zhai, E., Liu, X., Jin, X., and Zhou, J. Aegaeon: Effective GPU pooling for concurrent LLM serving on the market. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*, 2025. doi: 10.1145/3731569.3764815.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, 2022.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- Zheng, L., Yin, L., Xie, Z., Sun, C. L., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37: 62557–62583, 2024.
- Zheng, Z., Ren, X., Xue, F., Luo, Y., Jiang, X., and You, Y. Response length perception and sequence scheduling: an llm-empowered llm inference pipeline. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 193–210, 2024.

A. 完整版相关工作

本文关注不确定且具有混合计算与内存需求的请求级调度。下列研究方向大多与本文互补，可以与 *SageSched* 结合。

LLM 推理加速。 大量工作降低 LLM 推理的逐 *token* 计算与内存成本。在内核层面，FlashAttention 和 FlashAttention-2 重新组织注意力计算，改善内存局部性并减少冗余读写，获得更快、内存效率更高的精确注意力 (Dao et al., 2022; Dao, 2023)。模型压缩与低位宽量化在保持精度的同时进一步降低内存占用，例如 INT8 矩阵乘法和 Transformer 权重的训练后量化 (Dettmers et al., 2022; Frantar et al., 2022)。近期解码加速方法采用 *推测执行*：推测解码用快速草稿模型提出多个 *token*，再由目标模型验证 (Leviathan et al., 2023)；后续系统通过 *token* 树验证与在线机制进一步提高推测服务效率 (Miao et al., 2023; Liu et al., 2023)。

除 *token* 层面的加速外，系统设计还优化端到端流水线。预填充与解码分离，将以吞吐量为目标的预填充和对延迟敏感的解码解耦，并重构跨设备 KVCache 布局 (Zhong et al., 2024; Qin et al., 2024; Hu et al., 2024a;b)。分块预填充则将长上下文预填充与解码重叠，改善缓存复用，从而更好地管理吞吐量与延迟的权衡 (Agrawal et al., 2024; Hu et al., 2024b)。

这些技术提高单个请求的执行效率；*SageSched* 则在资源有限、请求长度变化的情况下决定接下来服务哪个请求，两者可以结合。

LLM 服务架构与复用。 许多系统通过重新设计批处理、准入控制和多租户资源共享，提高后端利用率。基础服务框架，例如 Orca 式连续批处理，以及 vLLM、SGLang 等现代引擎，强调较高设备利用率与高效 KVCache 管理 (Yu et al., 2022; Kwon et al., 2023; Zheng et al., 2024)。面向多模型、多租户场景，近期工作探索更细粒度的复用与资源池化：MuxServe 支持灵活的空间与时间复用，将多个 LLM 工作负载部署在共享 GPU 上 (Duan et al., 2024)；Aegaeon 为商用并发 LLM 服务提出有效的 GPU 池化方案 (Xiang et al., 2025)。

这些系统主要解决跨模型和租户的资源共享与放置；*SageSched* 则针对服务时间不确定时的后端内部请求排序，可作为每个 GPU 或资源池的调度器集成。

LLM 请求调度与公平性。 越来越多研究通过请求调度缓解队头阻塞并改善尾延迟。生产引擎通常从 FCFS 开始，该方法简单，但请求长度差异较大时可能严重损害 TTLT (Kwon et al., 2023; Zheng et al., 2024)。FastServe 采用类似 MLFQ 的策略，周期性降低优先级以近似 SRPT，在延迟收益与额外抢占开销之间权衡 (Wu et al., 2023)。较新的调度器采用先预测再排序： S^3 用微调的小模型估计生成长度 (Jin et al., 2023)；PO 查询 LLM，让它预估自身输出长度 (Zheng et al., 2023)；SSJF 使用代理模型预测输出长度，以近似 SJF (Qiu et al., 2024)；学习排序方法预测相对长度名次，而非精确长度 (Fu et al., 2024)。基于嵌入的调度器进一步利用从 LLM 层提取的提示特征，改善优先级分配 (Shahout et al., 2025)。

与此同时，感知公平性的服务研究形式化效率与差异化服务之间的矛盾，并提出满足公平目标的调度机制 (Sheng et al., 2024)。Llumnix 将运行时状态纳入调度，通过自适应重排改善服务质量 (Sun et al., 2024)。

相较上述工作，*SageSched* 有三点区别：(i) 感知输出长度的分布，而不是单点数值，避免脆弱的单值预测；(ii) 对 KVCache 增长造成的混合计算与内存成本建模，而非仅以输出长度作为服务成本；(iii) 在抢占决策中显式考虑 I/O 影响，力求在纠正优先级的同时避免过多交换。

KVCache 管理与长上下文服务。 解码反复关注所有已有 *token*，因此 KVCache 经常是主要内存瓶颈。vLLM 实现的 PagedAttention 将 KV 块分页，减少碎片并提高内存效率，从而改善 KVCache 分配 (Kwon et al., 2023)。除分配优化外，还可通过压缩或选择性保留减小 KVCache 大小：注意力汇点通过稳定注意力模式支持高效流式处理 (Xiao et al., 2023)；基于重度命中 *token* 的方法保留最重要的 *token*，在内存约束下近似完整注意力 (Zhang et al., 2023)。较新的工作随时间动态管理 KVCache：InfiniGen 提出用于高效生成式推理的动态 KVCache 管理 (Lee et al., 2024)；KIVI 无须调优，采用非对称 2 位量化处理 KV 缓存，降低内存压力与带宽需求 (Liu et al., 2024)。

这些改进提高可实现的并发量与每一步的执行效率；*SageSched* 与其互补，利用隐式反映 KVCache 增长和资源争用的成本分布设置请求优先级，进一步改善 TTLT。